

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

**MODUL VIII
PENGENALAN QUEUE**



Disusun Oleh :

NAMA : Rikza Nur Zaki

NIM : 103112430030

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

A. Dasar Teori Queue

Queue adalah struktur data linear yang bekerja dengan prinsip FIFO (First In, First Out), yaitu elemen yang masuk pertama akan keluar pertama, contohnya adalah antrian tiket, antrian kasir, antrian printer. Queue memiliki dua ujung, HEAD adalah tempat elemen diambil (dequeue), sedangkan TAIL adalah tempat elemen dimasukkan (enqueue)

B. Guided (berisi screenshot source code & output program disertai penjelasannya)

Guided 1

```
queue.h
#ifndef QUEUE_H
#define QUEUE_H

#define MAX_QUEUE 5

struct Queue
{
    int info[MAX_QUEUE];
    int head;
    int tail;
    int count;
};

void createQueue(Queue &Q);

bool isEmpty(Queue Q);

bool isFull(Queue Q);

void enqueue(Queue &Q, int x);

int dequeue(Queue &Q);

void printInfo(Queue Q);

#endif

queue.cpp
```

```

#include "queue.h"
#include <iostream>

using namespace std;

void createQueue(Queue &Q) {
    Q.head = 0;
    Q.tail = -1;
    Q.count = 0;
}

bool isEmpty(Queue Q) {
    return Q.count == 0;
}

bool isFull(Queue Q) {
    return Q.count == MAX_QUEUE;
}

void enqueue(Queue &Q, int x) {
    if (!isFull(Q)) {
        Q.tail = (Q.tail + 1) % MAX_QUEUE;
        Q.info[Q.tail] = x;
        Q.count++;
    } else {
        cout << "Antrean Penuh! " << endl;
    }
}

int dequeue(Queue &Q) {
    if (!isEmpty(Q)) {
        int x = Q.info[Q.head];
        Q.head = (Q.head + 1) % MAX_QUEUE;
        Q.count--;
        return x;
    } else {
        cout << "Antrean Kosong! " << endl;
        return -1;
    }
}

void printInfo(Queue Q) {
    if (isEmpty(Q)) {
        cout << "Isi Antrean: [Kosong]" << endl;
        return;
    }

    cout << "Isi Antrean: [";

    int i = Q.head;
    for (int n = 0; n < Q.count; n++) {
        cout << Q.info[i];
        if (n < Q.count - 1) cout << ", ";
        i = (i + 1) % MAX_QUEUE;
    }

    cout << "]" << endl;
}

```

main.cpp

```

#include "queue.h"
#include "queue.cpp"
#include <iostream>

using namespace std;

int main() {
    Queue Q;

    createQueue(Q);
    printInfo(Q);

    cout << "\n enqueue 3 elemen" << endl;
    enqueue(Q, 5);
    printInfo(Q);
    enqueue(Q, 2);
    printInfo(Q);
    enqueue(Q, 7);
    printInfo(Q);

    cout << "\n Dequeue 1 elemen" << endl;

    cout << "Elemen keluar: " << dequeue(Q) << endl;
    printInfo(Q);

    cout << "\n enqueue 1 elemen" << endl;
    enqueue(Q, 4);
    printInfo(Q);

    cout << "\n Dequeue 2 elemen" << endl;
    cout << "Elemen keluar: " << dequeue(Q) << endl;
    cout << "Elemen keluar: " << dequeue(Q) << endl;
    printInfo(Q);

    return 0;
}

```

Screenshots Output

```

Isi Antrean: [Kosong]

enqueue 3 elemen
Isi Antrean: [5]
Isi Antrean: [5, 2]
Isi Antrean: [5, 2, 7]

Dequeue 1 elemen
Elemen keluar: 5
Isi Antrean: [2, 7]

enqueue 1 elemen
Isi Antrean: [2, 7, 4]

Dequeue 2 elemen
Elemen keluar: 2
Elemen keluar: 7
Isi Antrean: [4]
PS D:\Kuliah\Struktur Data>

```

Deskripsi:

Ini adalah implementasi ADT Queue yang direpresentasikan menggunakan Circular Array berkapasitas 5 elemen, dengan melacak indeks awal antrian dan indeks elemen terakhir yang berputar secara modular di dalam array, serta menggunakan variabel count untuk secara eksplisit mengetahui jumlah elemen yang ada saat ini, sehingga kondisi kosong dan penuh dapat dideteksi dengan mudah dan operasi enqueue dan dequeue selalu

bergerak maju tanpa memerlukan pergeseran data.

- C. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)
Unguided 1

```
queue.h
#ifndef QUEUE_H
#define QUEUE_H

typedef int infotype;

struct Queue {
    infotype info[5];
    int head;
    int tail;
};

void createQueue(Queue &Q);
bool isEmptyQueue(Queue Q);
bool isFullQueue(Queue Q);
void enqueue(Queue &Q, infotype x);
infotype dequeue(Queue &Q);
void printInfo(Queue Q);

#endif
```

queue.cpp (alternatif 1)

```

#include <iostream>
#include "queue.h"
using namespace std;

void createQueue(Queue &Q){
    Q.head = 0;
    Q.tail = -1;
}

bool isEmptyQueue(Queue Q){
    return (Q.tail < Q.head);
}

bool isFullQueue(Queue Q){
    return (Q.tail == 4);
}

void enqueue(Queue &Q, infotype x){
    if (isFullQueue(Q)){
        cout << "Queue penuh" << endl;
    } else {
        Q.tail++;
        Q.info[Q.tail] = x;
    }
}

infotype dequeue(Queue &Q){
    if (isEmptyQueue(Q)){
        cout << "Queue kosong" << endl;
        return -1;
    } else {
        infotype x = Q.info[Q.head];
        for(int i = Q.head; i < Q.tail; i++){
            Q.info[i] = Q.info[i+1];
        }
        Q.tail--;
        return x;
    }
}

```

```

void printInfo(Queue Q){
    cout << Q.head << " - " << Q.tail << " | ";
    if(isEmptyQueue(Q)){
        cout << "empty queue" << endl;
    } else {
        for(int i=Q.head; i<=Q.tail; i++){
            cout << Q.info[i] << " ";
        }
        cout << endl;
    }
}

int main() {
    cout << "Hello World!" << endl;

    Queue Q;
    createQueue(Q);

    cout << "-----" << endl;
    cout << " H - T \t | Queue info" << endl;
    cout << "-----" << endl;

    printInfo(Q);
    enqueue(Q, 5); printInfo(Q);
    enqueue(Q, 2); printInfo(Q);
    enqueue(Q, 7); printInfo(Q);
    dequeue(Q);    printInfo(Q);
    enqueue(Q, 4); printInfo(Q);
    dequeue(Q);    printInfo(Q);
    dequeue(Q);    printInfo(Q);

    return 0;
}

```

queue.cpp (alternatif 2)

```

#include <iostream>
#include "queue.h"
using namespace std;

void createQueue(Queue &Q){
    Q.head = -1;
    Q.tail = -1;
}

bool isEmptyQueue(Queue Q){
    return (Q.head == -1);
}

bool isFullQueue(Queue Q){
    return (Q.tail == 4 && Q.head == 0);
}

void enqueue(Queue &Q, infotype x){
    if (isFullQueue(Q)){
        cout << "Queue penuh" << endl;
        return;
    }

    if (isEmptyQueue(Q)){
        Q.head = 0;
        Q.tail = 0;
    }
    else if (Q.tail == 4){
        int j = 0;
        for(int i=Q.head; i<=Q.tail; i++){
            Q.info[j++] = Q.info[i];
        }
        Q.head = 0;
        Q.tail = j-1;
        Q.tail++;
    }
    else {
        Q.tail++;
    }

    Q.info[Q.tail] = x;
}

```



```

infotype dequeue(Queue &Q){
    if(isEmptyQueue(Q)){
        cout << "Queue kosong" << endl;
        return -1;
    }

    infotype x = Q.info[Q.head];

    if(Q.head == Q.tail){
        Q.head = Q.tail = -1;
    } else {
        Q.head++;
    }

    return x;
}

void printInfo(Queue Q){
    cout << Q.head << " - " << Q.tail << " | ";
    if(isEmptyQueue(Q)){
        cout << "empty queue" << endl;
    } else {
        for(int i=Q.head; i<=Q.tail; i++){
            cout << Q.info[i] << " ";
        }
        cout << endl;
    }
}

int main() {
    cout << "QUEUE ALTERNATIF 2" << endl;

    Queue Q;
    createQueue(Q);

    cout << "-----" << endl;
    cout << " H - T \t | Queue info" << endl;
    cout << "-----" << endl;

    printInfo(Q);
    enqueue(Q, 5); printInfo(Q);
    enqueue(Q, 2); printInfo(Q);
    enqueue(Q, 7); printInfo(Q);
    dequeue(Q);   printInfo(Q);
    enqueue(Q, 4); printInfo(Q);
    dequeue(Q);   printInfo(Q);
    dequeue(Q);   printInfo(Q);

    return 0;
}

```

queue.cpp (alternatif 3)

```

#include <iostream>
#include "queue.h"
using namespace std;

void createQueue(Queue &Q){
    Q.head = -1;
    Q.tail = -1;
}

bool isEmptyQueue(Queue Q){
    return (Q.head == -1);
}

bool isFullQueue(Queue Q){
    return ((Q.tail + 1) % 5) == Q.head;
}

void enqueue(Queue &Q, infotype x){
    if(isFullQueue(Q)){
        cout << "Queue penuh" << endl;
        return;
    }

    if(isEmptyQueue(Q)){
        Q.head = 0;
        Q.tail = 0;
    } else {
        Q.tail = (Q.tail + 1) % 5;
    }

    Q.info[Q.tail] = x;
}

infotype dequeue(Queue &Q){
    if(isEmptyQueue(Q)){
        cout << "Queue kosong" << endl;
        return -1;
    }

    infotype x = Q.info[Q.head];

    if(Q.head == Q.tail){
        Q.head = Q.tail = -1;
    } else {
        Q.head = (Q.head + 1) % 5;
    }

    return x;
}

```

```

void printInfo(Queue Q){
    cout << Q.head << " - " << Q.tail << " | ";
    if(isEmptyQueue(Q)){
        cout << "empty queue" << endl;
        return;
    }

    int i = Q.head;
    while(true){
        cout << Q.info[i] << " ";
        if(i == Q.tail) break;
        i = (i + 1) % 5;
    }
    cout << endl;
}

int main() {
    cout << "QUEUE ALTERNATIF 3 (Circular)" << endl;

    Queue Q;
    createQueue(Q);

    cout << "-----" << endl;
    cout << " H - T \t | Queue info" << endl;
    cout << "-----" << endl;

    printInfo(Q);
    enqueue(Q, 5); printInfo(Q);
    enqueue(Q, 2); printInfo(Q);
    enqueue(Q, 7); printInfo(Q);
    dequeue(Q); printInfo(Q);
    enqueue(Q, 4); printInfo(Q);
    dequeue(Q); printInfo(Q);
    dequeue(Q); printInfo(Q);

    return 0;
}

```

Screenshots Output

queue (alternatif 1):

```

Hello World!
-----
 H - T | Queue info
-----
0 - -1 | empty queue
0 - 0 | 5
0 - 1 | 5 2
0 - 2 | 5 2 7
0 - 1 | 2 7
0 - 2 | 2 7 4
0 - 1 | 7 4
0 - 0 | 4

```

queue (alternatif 2):

```

Hello World!
-----
H - T | Queue info
-----
-1 - -1 | empty queue
0 - 0 | 5
0 - 1 | 5 2
0 - 2 | 5 2 7
1 - 2 | 2 7
1 - 3 | 2 7 4
2 - 3 | 7 4
3 - 3 | 4

```

queue (alternatif 3):

```

Hello World!
-----
H - T | Queue info
-----
-1 - -1 | empty queue
0 - 0 | 5
0 - 1 | 5 2
0 - 2 | 5 2 7
1 - 2 | 2 7
1 - 3 | 2 7 4
2 - 3 | 7 4
3 - 3 | 4

```

Deskripsi:

Untuk alternatif 1 pada bagian HEAD selalu berada di index 0 dan tidak bergerak, TAIL bertambah setiap insert, dan setiap kali delete semua elemen di belakang digeser satu langkah ke kiri sehingga antrean selalu dimulai dari index 0. Kemudian alternatif 2 untuk HEAD dan TAIL sama sama bergerak ke kanan mengikuti operasi insert dan delete, elemen hanya digeser ke kiri ketika TAIL sudah mencapai ujung array tetapi HEAD tidak di index 0 sehingga ruang kosong di depan harus dipakai lagi. Kemudian alternatif 3 untuk HEAD dan TAIL bergerak melingkar melalui array menggunakan modulus sehingga ketika mencapai ujung array akan kembali ke index 0 dan tidak pernah membutuhkan penggeseran elemen sama sekali.

D. Kesimpulan

Queue merupakan struktur data linear dengan prinsip FIFO (First In First Out) yang proses operasinya selalu dilakukan dari dua ujung, yaitu HEAD sebagai tempat pengambilan data (dequeue) dan TAIL sebagai tempat memasukkan data (enqueue). Implementasi queue menggunakan array dapat dilakukan dalam beberapa alternatif mekanisme pengelolaan indeks, yaitu Alternatif 1, Alternatif 2, dan Alternatif 3.

E. Referensi

Data Structures Using C & C++ ~ Yedidyah Langsam

Data Structures and Algorithm Analysis in C++ ~ Mark Allen Weiss